SystemVerilog · vérification

recto · classes, aléatoire, couverture, assertions

mot-clé type `macro $système // commentaire «à remplir» SV

Classes et objets

1 · classes

```
class Packet;
  bit [31:0] addr;              // public (défaut)
  local     int id;             // cette classe seule
  protected int crc;            // + classes filles
  static    int count = 0;      // partagé par tous
  function new(bit [31:0] a = 0);  // constructeur,
    addr = a;  count++;             // sans type retour
  endfunction
  virtual function void print();   // redéfinissable
    $display("addr=%h", addr);
  endfunction
  extern function int calc();  // corps hors classe
endclass
function int Packet::calc();   // corps de calc()
  return addr ^ crc;
endfunction
```

Héritage

```
class BigPacket extends Packet;
  rand bit [7:0] len;
  function new(bit [31:0] a = 0);
    super.new(a);               // constructeur parent
  endfunction
  virtual function void print();
    super.print();              // méthode du parent
    $display("len=%0d", len);
  endfunction
endclass
```

Handles, polymorphisme, cast

```
Packet    p;                    // handle = null
BigPacket b = new(32'h10);      // alloue l'objet
p = b;                          // fille -> parent : ok
p.print();                      // BigPacket::print
if (!$cast(b, p))               // parent -> fille :
  $error("mauvais type");       //   vérifié au run
n = Packet::count;              // membre statique
```

Pas de destructeur (ramasse-miettes) · `this` objet courant · `super` parent · `typedef class C;` déclaration anticipée.

Classe paramétrée

```
class Stack #(type T = int, int N = 8);
  T items[$:N-1];               // file bornée
  function void push(T v); items.push_back(v); endfunction
endclass
Stack #(string) s = new;
```

Métriques de couverture

Code	lignes / blocs, branches, conditions, toggle (0→1, 1→0), états et transitions des FSM. Automatique, dit ce qui a été *exécuté*	
Fonct.	`covergroup` écrits à partir du plan de vérif : dit quelles *fonctions* ont été exercées	
Assertions	`cover property` : scénarios atteints	
Code 100 %	Fonct. < 100 %	tests incomplets
Code < 100 %	Fonct. 100 %	plan de vérif incomplet
Code 100 %	Fonct. 100 %	bien, mais rien d'oublié ?

Exécuté ≠ vérifié : il faut aussi des checkers (assertions, scoreboard).
`covergroup` : colonne 3.

rand et contraintes

2 · aléatoire

```
typedef enum {LOW, MID, HIGH} kind_t;
class Trans;
  rand  bit [15:0] addr;
  rand  bit [31:0] data;
  rand  kind_t     kind;
  randc bit [3:0]  id;          // cyclique (permutation)
  rand  int        q[];         // taille aussi aléatoire
  constraint c_align { addr[1:0] == 2'b00; }
  constraint c_range {
    kind == LOW  -> addr inside {[0:15]};
    kind == HIGH -> addr inside {[128:255]};
  }
  constraint c_q { q.size() inside {[1:8]};
                   foreach (q[i]) q[i] < 100; }
endclass
```

Formes de contraintes

Relation	`a < b; a != 0;`	expression booléenne
Ensemble	`x inside {[0:9], 12};`	intervalles et valeurs
Exclusion	`!(x inside {[0:9]});`	hors de l'ensemble
Implication	`c -> x == 0;`	si c alors…
Condition	`if (c) {..} else {..}`	idem, avec else
Douce	`soft len < 20;`	abandonnée si conflit
Ordre	`solve k before a;`	tire k d'abord
Tableau	`foreach (q[i]) {...}`	par élément

Distribution : dist

```
x dist { [100:102] := 1,      // chaque valeur : poids 1
         200       := 2,      // poids 2
         [300:304] :/ 1 };    // poids 1 partagé sur 5
// total 3+2+1 = 6 : P(200) = 2/6, P(300..304) = 1/6
```

Tirage

```
Trans t = new;
if (!t.randomize())             // 0 si les contraintes
  $error("échec");              //   sont insolubles
assert (t.randomize() with {    // contraintes en ligne
  addr < 'h100;
  kind == local::kind;          // kind de l'appelant
});
void'(t.randomize());           // ignore le résultat
t.c_align.constraint_mode(0);   // coupe une contrainte
t.data.rand_mode(0);            // data n'est plus tirée
t.randomize(addr);              // ne tire que addr
```

`pre_randomize()` appelée juste avant le tirage (allouer un objet `rand`)
`post_randomize()` juste après (calculer les champs dérivés)

`$urandom()`	32 bits non signé
`$urandom_range(max, min)`	entre min et max
`std::randomize(x) with {...}`	hors classe

randcase : actions pondérées

```
randcase
  3    : x = 1;          // 3/8 si en = 1, 3/4 sinon
  1    : x = 2;          // 1/8 si en = 1, 1/4 sinon
  4*en : x = 3;          // poids = expression
endcase
```

La graine change les tirages : `vsim -sv_seed random`.

Couverture fonctionnelle

3 · couverture · SVA

```
covergroup cg_tr @(posedge clk);  // ou sample()
  cp_addr : coverpoint tr.addr {
    bins zero    = {0};
    bins low     = {[1:63], 65};  // 1 bin
    bins mid[4]  = {[64:127]};    // 4 bins égaux
    bins val[]   = {200,201,202}; // 1 bin / valeur
    ignore_bins  ign = {[250:255]};
    illegal_bins bad = {128};     // erreur si vu
  }
  cp_dir : coverpoint tr.dir;     // 1 bin / enum
  cx     : cross cp_addr, cp_dir;
endgroup
cg_tr cg = new;                   // instancier !
cg.sample();                      // échantillonner
cg.get_coverage();                // % atteint
```

Transitions et croisements

```
coverpoint state {
  bins t1   = (IDLE => RUN => DONE); // séquence
  bins t3   = (RUN [*3]);            // 3 fois de suite
  bins ts[] = (IDLE, RUN => DONE);   // 2 transitions
}
c : cross a, b {
  bins c1 = binsof(a.low) && binsof(b.zero);
  ignore_bins c2 = binsof(a) intersect {[100:200]};
}
```

sample() avec arguments (dans un monitor)

```
covergroup cg_par with function sample(bit p, byte d);
  cp_p : coverpoint p;
  cp_d : coverpoint ^d;           // parité des données
  cross cp_p, cp_d;
endgroup
cg_par cgp = new;  cgp.sample(cfg.parity, fr.data);
```

⚠ **Piège :** échantillonner ce qui est *observé* (monitor).

Assertions (SVA)

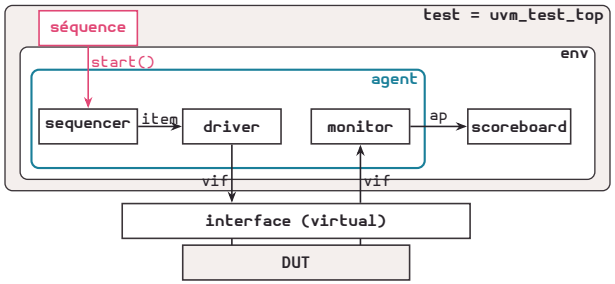
```
// immédiate : vérifiée quand elle est exécutée
assert (a == b) else $error("a=%0d b=%0d", a, b);
// concurrente : vérifiée à chaque front
property p_req_gnt;
  @(posedge clk) disable iff (!rst_n)
  req |-> ##[1:3] gnt;          // gnt 1 à 3 cycles
endproperty
A1: assert property (p_req_gnt) else $error("gnt");
C1: cover  property (p_req_gnt);  // compte les succès
assume property (@(posedge clk) !(push && pop));
sequence s_wr; req ##1 we ##1 !req; endsequence
bind fifo fifo_chk chk_i (.*);    // greffe dans le DUT
```

`a |-> b`	b au même cycle que a
`a |=> b`	b au cycle suivant (= `|-> ##1`)
`##n  ##[m:n]`	délai de n (m à n) cycles
`a[*n]  a[*m:n]`	a répété n fois consécutives
`a[->n]`	n-ième occurrence de a
`$rose(a) $fell(a)`	front montant / descendant
`$stable(a) $past(a,n)`	inchangé · valeur n cycles avant
`$onehot(v) $onehot0(v)`	exactement un 1 · au plus un 1
`$isunknown(v)`	v contient x ou z

`assume` : hypothèse sur les entrées (preuve formelle, Jasper).

Architecture d'un banc UVM

4 · architecture



`uvm_object`	items, séquences, config : créés pendant le test, sans parent
`uvm_component`	structure fixe : `new(name, parent)`, phases, hiérarchie

Phases

`build_phase`	func ↓	créer les fils, lire la config
`connect_phase`	func ↑	relier les ports TLM
`end_of_elaboration`	func ↑	`print_topology()`
`run_phase`	task ‖	tout le temps simulé
`reset` ··· `main`	task	sous-phases de run
`check` `report`	func ↑	vérifier, résumer

↓ du test vers les feuilles · ↑ l'inverse · ‖ tous les composants en parallèle.

Squelette d'un composant, objections

```
class my_test extends uvm_test;
  `uvm_component_utils(my_test)      // factory
  my_env env;                        // créé en build_phase
  function new(string name, uvm_component parent);
    super.new(name, parent);         // obligatoire
  endfunction
  task run_phase(uvm_phase phase);
    wr_rd_seq seq;
    phase.raise_objection(this);     // empêche la fin
    seq = wr_rd_seq::type_id::create("seq");
    void'(seq.randomize());
    seq.start(env.agt.sqr);          // lance la séquence
    phase.drop_objection(this);      // plus aucune
  endtask                            //  -> fin du test
endclass
```

Factory, config_db, rapports

```
drv = my_drv::type_id::create("drv", this);
it  = my_item::type_id::create("it");    // objet
// remplacer un type (dans le test, avant create)
base_drv::type_id::set_type_override(err_drv::get_type());
// set(contexte, chemin, clé, valeur) puis get
uvm_config_db#(int)::set(this, "env.agt*", "n", 4);
if (!uvm_config_db#(int)::get(this, "", "n", n))
  `uvm_fatal("CFG", "n absent")
`uvm_info("ID", $sformatf("x=%h", x), UVM_LOW)
```

`` `uvm_info ``	affiché si sa verbosité ≤ +UVM_VERBOSITY : UVM_NONE < LOW < MEDIUM < HIGH < FULL
`` `uvm_error ``	compté, la simulation continue (`` `uvm_warning `` idem, non bloquant)
`` `uvm_fatal ``	arrête la simulation

Transaction : sequence item

5 · stimulus

```
class apb_tr extends uvm_sequence_item;
  rand bit [31:0] addr, data;
  rand dir_t      dir;               // READ / WRITE
  `uvm_object_utils_begin(apb_tr)  // enregistre et
    `uvm_field_int(addr, UVM_ALL_ON | UVM_HEX)
    `uvm_field_enum(dir_t, dir, UVM_ALL_ON)
  `uvm_object_utils_end              // print, copy, compare
  function new(string name = "apb_tr");
    super.new(name);
  endfunction
endclass
```

Séquence

```
class wr_rd_seq extends uvm_sequence #(apb_tr);
  `uvm_object_utils(wr_rd_seq)
  rand int unsigned n;
  constraint c_n { n inside {[1:10]}; }
  task body();                       // le scénario
    bit [31:0] a;
    repeat (n) begin
      `uvm_do_with(req, { dir == WRITE; })
      a = req.addr;                  // relit la même
      `uvm_do_with(req, { dir == READ;
                          addr == local::a; })
    end
  endtask
endclass
```

`` `uvm_do(x) `` : crée, randomise, envoie l'item ou lance la sous-séquence x (`start_item` + `randomize` + `finish_item`). `new` : comme l'item. Lancée par le test : `seq.start(sqr)` (colonne 4).

Driver

```
class apb_drv extends uvm_driver #(apb_tr);
  `uvm_component_utils(apb_drv)
  virtual apb_if vif;                // accès aux signaux
  function void build_phase(uvm_phase phase);   // + new
    super.build_phase(phase);
    if (!uvm_config_db#(virtual apb_if)::get(
          this, "", "vif", vif))     // set par tb_top
      `uvm_fatal("NOVIF", "pas de vif")
  endfunction
  task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);  // bloquant
      drive(req);
      seq_item_port.item_done();         // item suivant
    end
  endtask
  task drive(apb_tr t);              // protocole APB, <=
    @(posedge vif.PCLK) vif.PSEL <= 1;  PADDR...
    @(posedge vif.PCLK) vif.PENABLE <= 1;
    do @(posedge vif.PCLK); while (!vif.PREADY);
    vif.PSEL <= 0;  vif.PENABLE <= 0;
  endtask
endclass
```

Monitor

6 · observation · assemblage

```
class apb_mon extends uvm_monitor;
  `uvm_component_utils(apb_mon)
  virtual apb_if vif;
  uvm_analysis_port #(apb_tr) ap;  // new() en build
  task run_phase(uvm_phase phase);
    apb_tr t;
    forever begin
      @(posedge vif.PCLK iff vif.PENABLE && vif.PREADY);
      t = apb_tr::type_id::create("t");
      t.addr = vif.PADDR;  ...
      ap.write(t);                 // à tous les abonnés
    end
  endtask
endclass
```

Scoreboard

```
`uvm_analysis_imp_decl(_apb)   // crée les classes
`uvm_analysis_imp_decl(_uart)  // uvm_analysis_imp_apb...
class my_scbd extends uvm_scoreboard;
  `uvm_component_utils(my_scbd)
  uvm_analysis_imp_apb  #(apb_tr,  my_scbd) apb_imp;
  uvm_analysis_imp_uart #(uart_fr, my_scbd) uart_imp;
  byte exp_q[$];                       // attendus (file)
  new : super.new() puis apb_imp = new("apb_imp", this)
  function void write_apb(apb_tr t);   // entrée
    if (t.dir == WRITE) exp_q.push_back(t.data[7:0]);
  endfunction
  function void write_uart(uart_fr f); // sortie
    byte e = exp_q.pop_front();
    if (f.data != e) `uvm_error("SCBD", "écart")
  endfunction
endclass
```

Assemblage : agent, env, top

```
// agent.build_phase
mon = apb_mon::type_id::create("mon", this);
if (get_is_active() == UVM_ACTIVE) begin
  sqr = apb_sqr::type_id::create("sqr", this);
  drv = apb_drv::type_id::create("drv", this);
end
// agent.connect_phase
drv.seq_item_port.connect(sqr.seq_item_export);
// env.connect_phase : monitors -> scoreboard
apb_agt.mon.ap.connect(scbd.apb_imp);
```

Module top

```
`include "uvm_macros.svh"
module tb_top;
  import uvm_pkg::*;  import my_pkg::*;
  logic clk = 0;  always #5 clk = ~clk;
  apb_if vif0 (clk);                   // interface
  dut u_dut (.PCLK(clk), .PADDR(vif0.PADDR), ...);
  initial begin
    uvm_config_db#(virtual apb_if)::set(null,
      "uvm_test_top.*", "vif", vif0);
    run_test();                        // +UVM_TESTNAME
  end
endmodule
```

```
vsim tb_top +UVM_TESTNAME=my_test -sv_seed random
```